GitHub Actions — Category 5: Advanced Workflow Patterns

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Categories 1–4 assumed. Jenkins pipeline experience assumed. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers → Full YAML → Gotchas ⚠ → Connections

---

Table of Contents

- 5.1 Matrix Builds — Static and Dynamic Matrix Generation
- 5.2 Concurrency Controls — Preventing Duplicate Runs, Queuing Deployments
- 5.3 Caching Strategies — actions/cache, Dependency Caching, Cache Keys
- 5.4 Artifacts — Uploading, Downloading, Cross-Job Sharing
- 5.5 Conditional Execution — if, always(), failure(), Step Conditions
- 5.6 Manual Approvals and workflow_dispatch with Inputs
- 5.7 Dynamic Workflow Generation — Generating YAML at Runtime
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

5.1 Matrix Builds — Static and Dynamic Matrix Generation ⚠

What It Is

A matrix strategy generates multiple parallel job instances from a set of variable combinations. Instead of writing N near-identical jobs, you declare one job and let GitHub Actions fan out the execution across all combinations.

Jenkins equivalent: Jenkins `parallel()` block with manually declared branches, or a loop that generates stages dynamically. The difference: GitHub Actions matrix is declarative and first-class — the parallelization is explicit in YAML and fully visible in the UI as separate job entries.

Why It Exists

Without matrix:

```
# 12 nearly-identical jobs just to test across Node versions and OS
combinations
```

```
  test-node18-ubuntu:
    runs-on: ubuntu-latest
    steps:
      - run: node --version  # 18
  test-node20-ubuntu:
    runs-on: ubuntu-latest
    steps:
      - run: node --version  # 20
  test-node18-windows: ...
  test-node20-windows: ...
  # ... 8 more
```

With matrix:

```
  # One job, 12 parallel instances declared in 5 lines
  strategy:
    matrix:
      node: [18, 20, 22]
      os: [ubuntu-latest, windows-latest, macos-latest]
      # GitHub Actions computes all 9 combinations automatically
```

How Matrix Works Internally

```
  Matrix definition evaluated at orchestrator level (before any runner
  starts)
          |
          v
  GitHub computes all combinations:
    {node: 18, os: ubuntu-latest}
    {node: 18, os: windows-latest}
    {node: 20, os: ubuntu-latest}
    ... etc.
          |
          v
  Each combination dispatched as a SEPARATE job to a SEPARATE runner
    -> Fully parallel (subject to your concurrency limit)
    -> Each has its own runner lifecycle, filesystem, environment
    -> Each appears as a separate entry in the workflow run UI
          |
          v
  If any combination fails:
    -> Default: ALL other running combinations continue
    -> fail-fast: true -> cancels all running combinations immediately
```

Static Matrix — Complete Reference

```yaml
name: Matrix Build Examples

on: [push, pull_request]

jobs:

  # ── Basic matrix ──────────────────────────────────────────────
  test-basic:
    strategy:
      matrix:
        node: [18, 20, 22]
        os: [ubuntu-latest, windows-latest, macos-latest]
        # 9 combinations total
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2
      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
# v4.2.0
        with:
          node-version: ${{ matrix.node }}
      - run: npm test

  # ── Matrix with include -- ADD combinations or properties ────────
  # include entries that match existing combinations ADD properties
  # include entries that match NO existing combination ADD new jobs
  test-with-include:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        node: [18, 20]
        # 4 base combinations
        include:
          # Add property to existing combination
          - os: ubuntu-latest
            node: 20
            experimental: true    # Adds 'experimental' property to this
combo only

          # Add a completely new combination not in the base
          - os: macos-latest
            node: 22
            experimental: true    # New job: macos + node22

    runs-on: ${{ matrix.os }}
    continue-on-error: ${{ matrix.experimental == true }}  # Don't fail if
experimental
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
```

```yaml
  # ── Matrix with exclude -- REMOVE specific combinations ──────────
  test-with-exclude:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node: [18, 20, 22]
        exclude:
          # Skip node 18 on macOS (no runner support reason)
          - os: macos-latest
            node: 18
          # Skip node 18 on Windows (legacy, not tested)
          - os: windows-latest
            node: 18
        # Result: 9 - 2 = 7 combinations
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version: ${{ matrix.node }}
      - run: npm test

  # ── Matrix with max-parallel -- throttle concurrency ──────────────
  test-throttled:
    strategy:
      fail-fast: false           # Don't cancel siblings when one fails
      max-parallel: 3            # Run at most 3 combinations
simultaneously
      matrix:
        service: [auth, payment, inventory, orders, notifications,
reporting]
        # 6 combinations, but max 3 run at once
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make test-${{ matrix.service }}

  # ── Matrix with custom objects ────────────────────────────────────
  # Each matrix entry is a full object -- powerful for complex configs
  deploy-environments:
    strategy:
      matrix:
        config:
          - env: staging
            region: us-east-1
            replicas: 2
            cluster: staging-eks
          - env: production-us
            region: us-east-1
            replicas: 5
            cluster: prod-eks-us
          - env: production-eu
            region: eu-west-1
```

```
            replicas: 5
            cluster: prod-eks-eu
    runs-on: ubuntu-latest
    environment: ${{ matrix.config.env }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - name: Deploy ${{ matrix.config.env }}
        run: |
          ./deploy.sh \
            --cluster ${{ matrix.config.cluster }} \
            --region ${{ matrix.config.region }} \
            --replicas ${{ matrix.config.replicas }}
```

Dynamic Matrix Generation ⚠

Dynamic matrix is the high-value interview topic: generating matrix values at runtime based on code changes, file content, or API responses — rather than hardcoding them in YAML.

Pattern 1: Matrix from Changed Files (Monorepo Service Detection)

```yaml
name: Dynamic Matrix from Changed Paths

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:

  # ── Step 1: Detect which services changed ──────────────────────
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      services: ${{ steps.detect.outputs.services }}
      has-changes: ${{ steps.detect.outputs.has-changes }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0    # Full history needed for git diff

      - name: Detect changed services
        id: detect
        run: |
          # Get list of changed files vs base branch (or previous commit)
          if [ "${{ github.event_name }}" = "pull_request" ]; then
            BASE="${{ github.event.pull_request.base.sha }}"
          else
            BASE="${{ github.event.before }}"
          fi
```

```
          CHANGED_FILES=$(git diff --name-only "${BASE}" HEAD 2>/dev/null
|| \
                          git diff --name-only HEAD~1 HEAD)

          echo "Changed files:"
          echo "$CHANGED_FILES"

          # Extract top-level service directories from changed files
          # services/auth/src/handler.go -> auth
          # services/payment/tests/test_main.py -> payment
          SERVICES=$(echo "$CHANGED_FILES" \
            | grep '^services/' \
            | cut -d'/' -f2 \
            | sort -u \
            | jq -R -s -c 'split("\n") | map(select(length > 0))')

          echo "Detected services: $SERVICES"

          if [ "$SERVICES" = "[]" ] || [ -z "$SERVICES" ]; then
            echo "services=[]" >> $GITHUB_OUTPUT
            echo "has-changes=false" >> $GITHUB_OUTPUT
          else
            echo "services=${SERVICES}" >> $GITHUB_OUTPUT
            echo "has-changes=true" >> $GITHUB_OUTPUT
          fi

  # ── Step 2: Use detected services as matrix ──────────────────────
  test-services:
    needs: detect-changes
    if: needs.detect-changes.outputs.has-changes == 'true'
    strategy:
      fail-fast: false
      matrix:
        service: ${{ fromJSON(needs.detect-changes.outputs.services) }}
        # fromJSON() converts the JSON string to an array
        # Each element becomes a separate matrix job
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Test ${{ matrix.service }}
        run: |
          cd services/${{ matrix.service }}
          make test

      - name: Build ${{ matrix.service }}
        run: |
          cd services/${{ matrix.service }}
          make build

  # ── Step 3: Deploy only changed services ──────────────────────────
  deploy-services:
    needs: [detect-changes, test-services]
```

```
    if: |
      github.event_name == 'push' &&
      needs.detect-changes.outputs.has-changes == 'true'
    strategy:
      fail-fast: false
      matrix:
        service: ${{ fromJSON(needs.detect-changes.outputs.services) }}
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: ./deploy.sh ${{ matrix.service }}
```

Pattern 2: Dynamic Matrix from JSON File

```
# services/matrix.json (committed to repo)
# {
#   "include": [
#     {"service": "auth",    "language": "go",     "go-version": "1.22"},
#     {"service": "payment", "language": "python", "python-version":
"3.12"},
#     {"service": "frontend","language": "node",   "node-version": "20"}
#   ]
# }

jobs:
  read-matrix:
    runs-on: ubuntu-latest
    outputs:
      matrix: ${{ steps.load.outputs.matrix }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - id: load
        run: |
          MATRIX=$(cat services/matrix.json | jq -c .)
          echo "matrix=${MATRIX}" >> $GITHUB_OUTPUT

  test-all:
    needs: read-matrix
    strategy:
      matrix: ${{ fromJSON(needs.read-matrix.outputs.matrix) }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - name: Test ${{ matrix.service }} (${{ matrix.language }})
        run: |
          cd services/${{ matrix.service }}
          make test
```

Pattern 3: Dynamic Matrix from API

```yaml
jobs:
  get-environments:
    runs-on: ubuntu-latest
    outputs:
      envs: ${{ steps.fetch.outputs.envs }}
    steps:
      - name: Fetch active environments from deployment API
        id: fetch
        run: |
          # Query your deployment API for active environments
          ENVS=$(curl -s \
            -H "Authorization: Bearer ${{ secrets.DEPLOY_API_TOKEN }}" \
            https://api.my-org.com/environments?status=active \
            | jq -c '[.[] | .name]')
          echo "envs=${ENVS}" >> $GITHUB_OUTPUT
        # ENVS might be: ["staging","canary","production-us","production-
eu"]

  deploy-all:
    needs: get-environments
    strategy:
      matrix:
        environment: ${{ fromJSON(needs.get-environments.outputs.envs) }}
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh ${{ matrix.environment }}
```

Pattern 4: Matrix with fromJSON and Complex Objects

```yaml
jobs:
  generate-matrix:
    runs-on: ubuntu-latest
    outputs:
      matrix: ${{ steps.gen.outputs.matrix }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - id: gen
        run: |
          # Build complex matrix objects programmatically
          MATRIX=$(python3 -c "
          import json, subprocess, os

          # Get changed services
          result = subprocess.run(
            ['git', 'diff', '--name-only', 'HEAD~1', 'HEAD'],
            capture_output=True, text=True
          )

          services = set()
          for f in result.stdout.strip().split('\n'):
```

```
            parts = f.split('/')
            if len(parts) >= 2 and parts[0] == 'services':
              services.add(parts[1])

          # Build matrix with service config from a manifest
          with open('services/manifest.json') as f:
            manifest = json.load(f)

          matrix_items = []
          for svc in sorted(services):
            if svc in manifest:
              matrix_items.append({
                'service': svc,
                **manifest[svc]    # merge service-specific config
              })

          print(json.dumps({'include': matrix_items}))
          ")
          echo "matrix=${MATRIX}" >> $GITHUB_OUTPUT

  run-matrix:
    needs: generate-matrix
    if: needs.generate-matrix.outputs.matrix != '{"include":[]}'
    strategy:
      matrix: ${{ fromJSON(needs.generate-matrix.outputs.matrix) }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: echo "Running ${{ matrix.service }} with config ${{
toJSON(matrix) }}"
```

Collecting Matrix Results

```
# Pattern: Fan-out (matrix) then Fan-in (collect results)
jobs:
  test:
    strategy:
      fail-fast: false
      matrix:
        service: [auth, payment, inventory]
    runs-on: ubuntu-latest
    outputs:
      # Matrix jobs can set outputs -- but only the LAST matrix run's
      # output is accessible. Use artifacts for multi-value collection.
      result: ${{ steps.run.outputs.result }}
    steps:
      - id: run
        run: |
          ./test.sh ${{ matrix.service }}
          echo "result=passed" >> $GITHUB_OUTPUT
```

```yaml
      - uses: actions/upload-artifact@v4
        with:
          name: test-results-${{ matrix.service }}   # Unique name per
matrix job
          path: test-results/

  # Fan-in: collect all matrix results
  report:
    needs: test
    if: always()    # Run even if some matrix jobs failed
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          pattern: test-results-*   # Download ALL matching artifacts
          merge-multiple: true
      - run: |
          # All test result files are now in current directory
          ./generate-combined-report.sh
          cat combined-report.md >> $GITHUB_STEP_SUMMARY
```

Interview Answer (30-45 seconds)

> "Matrix strategy fans out a single job definition into N parallel jobs across a combination of variables. You declare the matrix dimensions — OS versions, language versions, environments — and GitHub computes all combinations and runs each as an independent job on its own runner. `include` adds properties to existing combinations or entirely new combinations. `exclude` removes specific combinations. `fail-fast: false` is important for test matrices — you want all results even if one combination fails. Dynamic matrix is where it gets powerful: a first job detects which services changed via `git diff`, serializes the list as JSON to an output, and a downstream job uses `fromJSON()` to feed that as the matrix. This pattern runs tests only for the services that actually changed in a monorepo, rather than rebuilding everything."

Gotchas ⚠

- **Matrix output values are only the LAST matrix job's output.** If three parallel matrix jobs all set `outputs.result`, the `needs.job.outputs.result` you read downstream is only from whichever matrix instance happened to finish last. Use artifacts for reliable multi-value collection.
- **`fromJSON()` requires valid JSON.** If the upstream step outputs an empty string, invalid JSON, or `null`, `fromJSON()` throws an error that can be confusing to debug. Always validate the JSON before outputting.
- **Empty matrix causes a workflow error.** If `fromJSON(needs.detect.outputs.services)` resolves to `[]`, the matrix job fails with "Matrix vector is empty." Guard with `if: needs.detect.outputs.has-changes == 'true'` or check for empty array.
- **`include` behavior is non-obvious.** An `include` entry with ALL keys matching an existing combination ADDS properties to that combination. An `include` entry with any key NOT matching

creates a new combination. Partial matches (some keys match, some don't) create a new combination with the matching original combination as the base — this surprises people.

- **Max combinations is 256 per matrix.** GitHub enforces a hard limit. For large monorepos, you may need to chunk your matrix into multiple workflow runs or use a different parallelization strategy.
- **`continue-on-error` at the matrix job level vs `fail-fast: false` are different.** `fail-fast: false` lets all siblings continue when one fails. `continue-on-error: true` makes the job appear as successful even when it fails — don't use this unless you truly want failures silenced.

5.2 Concurrency Controls — Preventing Duplicate Runs, Queuing Deployments

---

What It Is

The `concurrency:` key defines a named group for workflow runs or individual jobs. When a new run starts for a group, GitHub can either cancel the existing run (cancel-in-progress) or queue the new run until the existing one finishes.

Jenkins equivalent: Jenkins' `disableConcurrentBuilds()` option in Declarative pipelines, or throttle-concurrent-builds plugin. The difference: GitHub's concurrency model is per-named-group, not per-job-type, and the group name is an expression you control — enabling powerful targeting.

Why It Exists

Problems without concurrency control:

```
Scenario 1: Multiple PR pushes trigger duplicate CI runs
  Developer pushes commit 1 -> CI run starts
  Developer fixes typo -> pushes commit 2 -> second CI run starts
  Both run for 8 minutes
  Only the latest result matters -- run 1 is wasted compute

Scenario 2: Race condition in production deploys
  Two merges to main 30 seconds apart
  Two deploy jobs run simultaneously
  Deploy A writes config version 1
  Deploy B writes config version 2
  Deploy A finishes writing (overwrites B's partial changes)
  Production is in an inconsistent state

Scenario 3: Environment resource contention
  Five PRs merge simultaneously
  Five staging deploys run concurrently
  They all try to update the same Helm release
  Four fail with "release already in progress"
```

How Concurrency Works Internally

```
New workflow run starts for group "ci-my-repo-main"
        |
        v
GitHub checks: is there an active run in this group?
        |
        |-- NO: Start the new run normally
        |
        `-- YES: Check cancel-in-progress
              |
              |-- cancel-in-progress: true
              |     -> Cancel the existing run
              |     -> Start the new run immediately
              |
              `-- cancel-in-progress: false (default)
                    -> Queue the new run
                    -> New run waits until existing run completes
                    -> Only ONE queued run is kept
                    -> If ANOTHER run arrives while waiting,
                       the waiting run is CANCELLED (not queued behind)
                    -> The newest run always wins the queue slot
```

Key insight: With `cancel-in-progress: false`, you get "run latest only" semantics for the queue — not FIFO ordering. This means if three runs pile up, only the newest one executes after the current finishes.

Concurrency Patterns — Complete Reference

```yaml
name: Concurrency Examples

on:
  push:
    branches: [main, 'feature/**']
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]

jobs:

  # ── Pattern 1: Cancel duplicate CI runs per branch ──────────────
  # Each branch has its own concurrency group
  # New push cancels the previous run for that branch
  ci:
    runs-on: ubuntu-latest
```

```yaml
    concurrency:
      group: ci-${{ github.repository }}-${{ github.ref }}
      cancel-in-progress: true    # New push cancels in-progress run
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: npm test

  # ── Pattern 2: Queue deploys (never cancel -- ensure completion) ─
  # Only one deploy to staging at a time
  # New deploys wait for the current one to finish
  deploy-staging:
    runs-on: ubuntu-latest
    environment: staging
    concurrency:
      group: deploy-staging-${{ github.repository }}
      cancel-in-progress: false   # Queue: don't cancel ongoing deploys
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: ./deploy.sh staging

  # ── Pattern 3: Environment-scoped concurrency ────────────────────
  # Separate concurrency per environment -- staging and production
independent
  deploy-any-env:
    runs-on: ubuntu-latest
    concurrency:
      group: deploy-${{ github.event.inputs.environment }}-${{
github.repository }}
      cancel-in-progress: false
    environment: ${{ github.event.inputs.environment }}
    steps:
      - run: ./deploy.sh ${{ github.event.inputs.environment }}

  # ── Pattern 4: Workflow-level concurrency (all jobs affected) ────
  # Defined at top level -- applies to the entire workflow run
  # Used for overall pipeline deduplication

  # ── Pattern 5: PR-specific concurrency ───────────────────────────
  # Group by PR number -- different PRs run in parallel
  # Same PR cancels previous run when new commit pushed
  pr-ci:
    runs-on: ubuntu-latest
    concurrency:
      group: pr-ci-${{ github.event.pull_request.number || github.ref }}
      # github.event.pull_request.number is null for push events
      # fallback to github.ref for non-PR events
      cancel-in-progress: true
    steps:
      - run: npm test
```

Workflow-Level vs Job-Level Concurrency

```yaml
# ── Workflow-level: applies to the entire workflow run ──────────
name: Deploy Pipeline
on: push

# This group controls the entire workflow -- if a new push comes in
# while this workflow is running, behavior depends on cancel-in-progress
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}
  # Cancel in-progress for feature branches (fast feedback)
  # Queue for main (never cancel a main deploy)

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: make build

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: production
    # Job-level concurrency can ADD a more restrictive constraint
    concurrency:
      group: production-deploy-${{ github.repository }}
      cancel-in-progress: false   # Never cancel ongoing production deploy
    steps:
      - run: ./deploy.sh
```

Real-World: Full Pipeline with Thoughtful Concurrency

```yaml
name: Full Pipeline with Concurrency

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

# Cancel duplicate CI for feature branches; queue for main
concurrency:
  group: pipeline-${{ github.repository }}-${{ github.ref }}
  cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}

jobs:

  build-and-test:
    runs-on: ubuntu-latest
    # Job-level: cancel duplicate test runs within same group
```

```yaml
    concurrency:
      group: test-${{ github.ref }}-${{ github.sha }}
      cancel-in-progress: true
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make test

  deploy-staging:
    needs: build-and-test
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: staging
    # Queue staging deploys -- one at a time, never cancel
    concurrency:
      group: deploy-staging-${{ github.repository }}
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: ./deploy.sh staging

  deploy-production:
    needs: deploy-staging
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    # Queue production deploys -- absolute: never cancel, never race
    concurrency:
      group: deploy-production-${{ github.repository }}
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: ./deploy.sh production
```

Interview Answer (30-45 seconds)

> "Concurrency controls define a named group for runs or jobs, and control what happens when a new run arrives while another is active in that group. The two behaviors are: `cancel-in-progress: true`, which cancels the existing run and starts the new one — ideal for CI where only the latest result matters. And `cancel-in-progress: false`, which queues the new run and lets the current one finish — mandatory for deployments where you cannot interrupt a running deploy. The group name is an expression, which is the power: `deploy-staging-${{ github.repository }}` scopes the queue to a specific environment, `pr-ci-${{ github.event.pull_request.number }}` gives each PR its own concurrency group so different PRs run in parallel. One key subtlety: with queuing, only one run waits — if three more arrive while one is running, the first two waiting are cancelled; only the newest waits."

Gotchas ⚠

- **Only ONE run waits in the queue.** If a run is active and three more queue up, runs 2 and 3 are cancelled when run 4 arrives. The queue is "latest-wins," not FIFO. This is correct behavior for CI/CD but surprises people expecting all runs to execute.
- **Concurrency at workflow level vs job level stack independently.** A workflow-level group and a job-level group create two separate constraints. A job only executes when BOTH its own concurrency group AND the workflow-level group allow it.
- **Cancelling a run that's waiting for environment approval** is valid. If a pending approval deploy gets cancelled by a newer run, the reviewer sees the pending approval disappear — which can be confusing. Communicate this behavior to approvers.
- **`github.ref` for PRs is `refs/pull/123/merge` not the source branch.** If you use `${{ github.ref }}` in a concurrency group for both push and PR events, they'll have different group names for the same branch. Use `${{ github.event.pull_request.number || github.ref }}` for consistent grouping.
- **Concurrency groups are global across the workflow, not per-repo-unique by default.** If two different repos both use `group: deploy-staging`, they share the concurrency group. Always include `${{ github.repository }}` in group names.

5.3 Caching Strategies — actions/cache, Dependency Caching, Cache Keys ⚠

What It Is

`actions/cache` saves and restores files between workflow runs using a content-addressed key. The primary use case is caching dependency directories so subsequent runs skip downloading packages already downloaded in a previous run.

Jenkins equivalent: Jenkins' built-in workspace persistence (jobs run on the same agent keep files between runs) or explicit stash/unstash. The difference: GitHub-hosted runners start fresh every run with empty filesystems — there is NO workspace persistence. Caching is the explicit mechanism for anything you want to preserve across runs.

Why Caching Matters

```
Without cache (typical npm project, cold run):
  npm ci: download 847 packages = 45 seconds

With warm cache hit:
  Restore cache: 3 seconds
  npm ci (packages already in node_modules / npm cache): 4 seconds
  Total: 7 seconds  --  6x faster

For a team running 50 PRs/day:
  Without cache: 50 x 45s = 37.5 minutes of install time per day
```

```
  With cache:    50 x  7s =  5.8 minutes of install time per day
  Saving: ~30 minutes/day / ~15 CI runner-minutes/day at scale
```

How actions/cache Works Internally

```
SAVE phase (post-job, after all steps complete):
  1. Check: did a cache MISS occur during this run?
  2. If yes: tar the specified paths, upload to GitHub's cache storage
  3. Store under the exact key that was requested
  4. Cache TTL: 7 days of non-access, max 10 GB per repo

RESTORE phase (before job steps execute):
  1. Look up exact key in cache storage
  2. EXACT HIT: download and extract, set cache-hit=true
  3. EXACT MISS: check restore-keys prefixes in order
  4. PREFIX HIT: download closest match, set cache-hit=false
     (run still proceeds, new cache saved at end with exact key)
  5. FULL MISS: no restore, run proceeds, new cache saved at end
```

Cache Key Design — The Critical Skill ⚠

Cache keys must balance two competing requirements:

- **Specificity:** different dependency sets must have different keys (no stale cache)
- **Reuse:** commits that don't change dependencies must reuse existing cache

```
GOOD cache key formula:
  ${{ runner.os }}-<ecosystem>-${{ hashFiles('<lockfile>') }}

  runner.os      -> different OS = different binary packages = different
cache
  ecosystem      -> prevents npm and pip caches colliding on same runner
  hashFiles()    -> changes ONLY when lockfile changes = stable cache
across commits
```

```yaml
# ── npm / Node.js ──────────────────────────────────────────────
- uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8  # v4.1.1
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-

# ── pip / Python ───────────────────────────────────────────────
- uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
```

```yaml
    with:
      path: ~/.cache/pip
      key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
      restore-keys: |
        ${{ runner.os }}-pip-

  # ── Maven / Java ──────────────────────────────────────────────
  - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
    with:
      path: ~/.m2/repository
      key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
      restore-keys: |
        ${{ runner.os }}-maven-

  # ── Gradle / Java ─────────────────────────────────────────────
  - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
    with:
      path: |
        ~/.gradle/caches
        ~/.gradle/wrapper
      key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle*', '**/gradle-
  wrapper.properties') }}
      restore-keys: |
        ${{ runner.os }}-gradle-

  # ── Go modules ────────────────────────────────────────────────
  - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
    with:
      path: |
        ~/go/pkg/mod
        ~/.cache/go-build
      key: ${{ runner.os }}-go-${{ hashFiles('**/go.sum') }}
      restore-keys: |
        ${{ runner.os }}-go-

  # ── Rust / Cargo ──────────────────────────────────────────────
  - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
    with:
      path: |
        ~/.cargo/bin/
        ~/.cargo/registry/index/
        ~/.cargo/registry/cache/
        ~/.cargo/git/db/
        target/
      key: ${{ runner.os }}-cargo-${{ hashFiles('**/Cargo.lock') }}
      restore-keys: |
        ${{ runner.os }}-cargo-

  # ── Ruby / Bundler ────────────────────────────────────────────
  - uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
    with:
      path: vendor/bundle
      key: ${{ runner.os }}-gems-${{ hashFiles('**/Gemfile.lock') }}
```

```
      restore-keys: |
        ${{ runner.os }}-gems-
```

Using Built-In Caching in setup-* Actions (Simpler)

Many `setup-*` actions have built-in caching that handles key generation automatically:

```
- uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e  # v4
  with:
    node-version: '20'
    cache: 'npm'       # Automatically caches ~/.npm with correct key
    # Also supports: 'yarn', 'pnpm'

- uses: actions/setup-python@v5
  with:
    python-version: '3.12'
    cache: 'pip'        # Caches ~/.cache/pip automatically

- uses: actions/setup-go@v5
  with:
    go-version: '1.22'
    cache: true         # Caches ~/go/pkg/mod and build cache

- uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: 'temurin'
    cache: 'maven'      # Or 'gradle', 'sbt'
```

Prefer the built-in cache over manual `actions/cache` when a setup action supports it — it handles edge cases and key generation for you.

Advanced Caching Patterns

Pattern 1: Multi-Level Cache Keys

```
# Level 1: Exact match on lockfile hash (most specific)
# Level 2: OS + week number (broad fallback -- partial reuse)
# Level 3: OS only (last resort partial reuse)

- uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
  id: cache
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-v3-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-v3-${{ hashFiles('**/package-lock.json') }}
```

```
      ${{ runner.os }}-npm-v3-
      ${{ runner.os }}-npm-

# Check if cache was hit
- if: steps.cache.outputs.cache-hit != 'true'
  run: npm ci        # Full install on miss
- if: steps.cache.outputs.cache-hit == 'true'
  run: npm ci --prefer-offline  # Faster install leveraging cache
```

Pattern 2: Docker Layer Caching

```
jobs:
  docker-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-
action@c47758b77c9736f4b2ef4073d4d51994fabfe349  # v3

      # Cache Docker layers in GitHub Cache (or registry -- see below)
      - name: Build and push with layer caching
        uses: docker/build-push-
action@4f58ea79222b3b9dc2c8bbdd6debcef730109a75  # v6
        with:
          push: false
          cache-from: type=gha              # Read from GitHub Actions
cache
          cache-to: type=gha,mode=max       # Write all layers to cache
          tags: myapp:${{ github.sha }}

      # Alternative: use registry cache (more persistent, no 10GB limit)
      - name: Build with registry cache
        uses: docker/build-push-
action@4f58ea79222b3b9dc2c8bbdd6debcef730109a75
        with:
          push: true
          cache-from: type=registry,ref=ghcr.io/my-org/myapp:buildcache
          cache-to: type=registry,ref=ghcr.io/my-
org/myapp:buildcache,mode=max
          tags: ghcr.io/my-org/myapp:${{ github.sha }}
```

Pattern 3: Branch-Aware Cache Isolation

```
# Problem: feature branch should not pollute main's cache
# Solution: branch-specific key, fallback to main's cache

- uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
```

```
    with:
      path: ~/.npm
      key: ${{ runner.os }}-npm-${{ github.ref_name }}-${{
hashFiles('**/package-lock.json') }}
      restore-keys: |
        ${{ runner.os }}-npm-${{ github.ref_name }}-
        ${{ runner.os }}-npm-main-     # Fall back to main branch cache
        ${{ runner.os }}-npm-
```

Pattern 4: Post-Job Cache Save Control

```
# actions/cache v4 supports save-always and lookup-only flags

- uses: actions/cache/restore@v4   # Separate restore step
  id: restore-cache
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}

- run: npm ci

# Only save cache if tests PASSED (don't cache broken state)
- uses: actions/cache/save@v4
  if: success()   # Only on step success
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
```

Pattern 5: Turbo / Nx Build Caching (Monorepo)

```
# For monorepos using Turborepo or Nx, the build tool has its own cache
# GitHub Actions cache serves as the persistent store for the build tool
cache

- uses: actions/cache@3624ceb22c1c5a301c8db4169662070a689d9ea8
  with:
    path: .turbo    # Turborepo local cache directory
    key: ${{ runner.os }}-turbo-${{ github.sha }}
    restore-keys: |
      ${{ runner.os }}-turbo-

- name: Build affected packages
  run: npx turbo run build --filter=[HEAD^1]
  # Turbo reads from .turbo cache, only rebuilds what changed
```

Interview Answer (30-45 seconds)

> "GitHub-hosted runners start with an empty filesystem every run — there's no Jenkins-style workspace persistence. `actions/cache` is the explicit mechanism for preserving files between runs. It works in two phases: restore at the start of the job (before steps), and save at the end (after steps). The cache key is the critical design decision — it must be specific enough that dependency changes invalidate it, but stable enough across commits that unrelated changes don't bust it. The formula is: OS + ecosystem + hashFiles(lockfile). `hashFiles()` returns a hash of the lockfile content — changes only when dependencies change. For partial reuse, `restore-keys` provides fallback prefixes. Most setup actions now have built-in caching that handles key design automatically — prefer those over manual cache configuration."

Gotchas ⚠

- **The save step only runs if the key was a MISS.** If the exact key already exists in cache, the save step is skipped — GitHub doesn't overwrite existing cache entries with the same key. To force a refresh, change the key (e.g., add a version suffix like `npm-v2-`).
- **Cache is NOT shared between repos.** Cache is scoped to the repository. A monorepo's sub-packages all share the same cache pool. Different repos cannot share caches.
- **Cache is NOT available across forks.** Fork PRs can read the parent repo's cache but cannot write to it. Their saves go to the fork's own cache pool.
- **`hashFiles()` globs are evaluated from the workspace root.** Make sure the pattern resolves to the correct files. `**` is recursive. `hashFiles('**/package-lock.json')` hashes ALL package-lock.json files in all subdirectories.
- **10 GB repo cache limit.** When the limit is reached, GitHub evicts the least recently accessed entries. Cache entries also expire after 7 days of non-access. For Docker layer caches, prefer registry-based caching which doesn't consume the repo limit.
- **Cache restore adds time regardless of hit or miss.** The lookup itself takes 1-2 seconds. For very short jobs (under 30 seconds total), the overhead may outweigh the benefit.
- **Don't cache the `node_modules` directory directly.** Cache `~/.npm` (npm's package cache) instead. `node_modules` is large, OS-specific, and platform-tied. After restoring the npm cache, `npm ci` still runs but completes fast because packages are already in the local cache.

5.4 Artifacts — Uploading, Downloading, Cross-Job Sharing

What It Is

Artifacts are files produced by a workflow job that need to be shared with other jobs, preserved for download after the run completes, or passed to downstream workflows. Unlike cache (which is keyed and reused across runs), artifacts are per-run outputs — each run produces fresh artifacts.

Jenkins equivalent: `archiveArtifacts` for post-build artifacts, `stash`/`unstash` for cross-stage file sharing. The difference: GitHub artifacts are explicitly uploaded/downloaded by name — there's no implicit workspace sharing between jobs (because each job runs on a separate runner).

Upload and Download — Core Usage

```yaml
name: Artifact Examples

jobs:

  # ── Producer job: build and upload ──────────────────────────
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Build application
        run: |
          make build
          # Produces: dist/ directory and coverage/lcov.info

      # Upload build output (default retention: 90 days)
      - uses: actions/upload-
artifact@65c4c4a1ddee5b72f698c2d2d51cc4b0c2b64e41  # v4.6.1
        with:
          name: build-dist-${{ github.sha }}     # Unique name per run
          path: dist/
          retention-days: 30                     # Override default
retention
          compression-level: 9                   # 0-9, default 6
          if-no-files-found: error               # error | warn | ignore
(default warn)

      # Upload multiple paths
      - uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: coverage-reports
          path: |
            coverage/
            test-results/*.xml
            !coverage/tmp/           # Exclude pattern with !

  # ── Consumer job: download and use ──────────────────────────
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      # Download specific artifact by name
      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16  # v4.1.8
        with:
          name: build-dist-${{ github.sha }}
          path: ./dist    # Where to extract it (default: current
directory)
```

```
      - run: ./deploy.sh dist/

  # ── Download ALL artifacts from this run ──────────────────────
  report:
    needs: [build, test-unit, test-e2e]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          path: all-artifacts/    # Downloads ALL artifacts into subdirs
by name
          # Each artifact extracted into all-artifacts/<artifact-name>/

      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          pattern: test-results-*   # Download all matching artifact names
          merge-multiple: true      # Merge into single directory (not
subdirs)
          path: merged-results/
```

Artifact Patterns for Cross-Job Sharing

```
# Pattern: Build once, deploy many ways
# Critical: avoids re-building the artifact in each downstream job

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      image-tag: ${{ steps.build.outputs.tag }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - id: build
        run: |
          make build
          echo "tag=${{ github.sha }}" >> $GITHUB_OUTPUT
      - uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: app-binary
          path: |
            bin/app
            Dockerfile
            helm/

  test-integration:
    needs: build
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          name: app-binary
      - run: ./bin/app --test-mode

  security-scan:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          name: app-binary
      - run: trivy fs ./  # Scan the binary

  deploy:
    needs: [test-integration, security-scan]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          name: app-binary
      - run: docker build -t myapp:${{ needs.build.outputs.image-tag }} .
```

Artifacts vs Cache vs Outputs — When to Use What

Mechanism	Use When	Scope	Expires
Artifact	Passing files between jobs; preserving build outputs for download	Per-run	1–90 days
Cache	Reusing dependencies across multiple runs	Per-repo, keyed	7 days LRU
Job outputs	Passing small values (strings, numbers, JSON < 50KB) between jobs	Per-run	Run lifetime
Env vars	Passing values between steps within one job	Per-job-step	Step lifetime

---

Downloading Artifacts from Other Workflow Runs

```
# Download artifacts from a DIFFERENT workflow run (e.g., a release
workflow)
# Requires actions: read permission
```

```
- uses: actions/download-artifact@fa0a91b85d4f404e444e00e005971372dc801d16
# v4
  with:
    name: release-binary
    run-id: ${{ github.event.workflow_run.id }}   # From workflow_run
trigger
    github-token: ${{ secrets.GITHUB_TOKEN }}

# Or download from a specific run ID
- uses: actions/download-artifact@fa0a91b85d4f404e444e00e005971372dc801d16
  with:
    name: build-output
    run-id: 1234567890   # Specific run ID
    repository: my-org/my-repo  # Can be different repo (needs token with
repo access)
    github-token: ${{ secrets.GH_PAT }}
```

Interview Answer (30-45 seconds)

> "Artifacts are per-run files uploaded from one job and downloaded by another — the explicit cross-job file sharing mechanism. Since each GitHub Actions job runs on a separate runner with a clean filesystem, there's no implicit workspace sharing like Jenkins stash/unstash. You upload with a unique name using `actions/upload-artifact`, and downstream jobs download by that name using `actions/download-artifact`. Key design principle: build once in a producer job, upload the artifact, and have all downstream consumers download the same artifact — avoid rebuilding in each job. Artifacts differ from cache: cache is keyed and reused across runs to speed up recurring operations; artifacts are run-specific outputs retained for inspection or consumption. Retention defaults to 90 days, configurable down to 1 day."

Gotchas ⚠

- **Artifact names must be unique within a workflow run.** If two jobs upload an artifact with the same name, the second upload fails. For matrix jobs, include the matrix variable in the name: `test-results-${{ matrix.service }}`.
- **v4 breaking changes from v3.** `actions/upload-artifact@v4` changed behavior: uploading to an existing artifact name within the same run now merges (by default) or errors. The `merge-multiple: true` option in download-artifact v4 handles this explicitly.
- **Artifacts are not available between jobs if the upload job is cancelled.** If the producer job is cancelled (e.g., by concurrency cancellation), downstream jobs trying to download will fail with "artifact not found."
- **Large artifacts cause significant overhead.** Uploading 500MB on every run adds minutes to pipeline time. For large build outputs (Docker images), prefer pushing directly to a registry rather than uploading as an artifact.
- **The `path:` in download-artifact is where the artifact is extracted, NOT a filter.** If you specify `path: ./my-dir`, the entire artifact is extracted into `./my-dir`. To filter specific files, download everything and use shell commands to select what you need.

- **Default retention is 90 days but configurable.** Org and repo settings can set a maximum retention limit. A workflow requesting `retention-days: 90` on an org with a 30-day max gets 30 days silently.

5.5 Conditional Execution — if, always(), failure(), Step Conditions

---

What It Is

GitHub Actions provides conditional expression evaluation at both the job level and the step level via the `if:` key. These conditions control whether a job or step executes, using the expression syntax and a set of status check functions.

Jenkins equivalent: `when {}` directives in Declarative Pipeline, or `if` statements in Scripted Pipeline. The difference: GitHub's status functions (`success()`, `failure()`, `always()`, `cancelled()`) have specific semantics tied to the current execution context that differ from Jenkins' `currentBuild.result` checks.

Status Check Functions — Complete Reference

```
success()  -- Returns true if ALL previous steps/jobs completed
successfully
              This is the DEFAULT if no if: condition is specified
              A step with no if: runs only on success

failure()  -- Returns true if ANY previous step/job FAILED
              Does NOT trigger for cancelled runs

always()   -- Always returns true regardless of status
              Runs even if previous steps failed OR were cancelled

cancelled()-- Returns true if the workflow/job was cancelled
```

```yaml
jobs:
  demo:
    runs-on: ubuntu-latest
    steps:
      - run: ./might-fail.sh      # Step 1

      - run: ./normal-step.sh     # Skipped if step 1 fails (default:
success())

      - if: failure()             # Runs ONLY if a previous step failed
```

```
        run: ./handle-failure.sh

      - if: always()              # ALWAYS runs (success OR failure OR
cancel)
        run: ./cleanup.sh

      - if: success()             # Explicit -- same as default (no if:)
        run: ./post-success.sh

      - if: cancelled()           # Runs only if workflow was cancelled
        run: ./handle-cancel.sh

      # Combining status + conditions
      - if: failure() && github.ref == 'refs/heads/main'
        run: ./alert-main-branch-failure.sh
```

Job-Level Conditions

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: make build

  # Run only on main branch pushes (not PRs, not feature branches)
  deploy:
    needs: build
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh

  # Run only if build FAILED (notification job)
  notify-failure:
    needs: build
    if: failure()   # Means: if the 'build' job failed
    runs-on: ubuntu-latest
    steps:
      - run: ./send-alert.sh

  # Run regardless of build status (cleanup/reporting)
  always-report:
    needs: [build, deploy]
    if: always()    # Run even if build or deploy failed
    runs-on: ubuntu-latest
    steps:
      - run: ./generate-report.sh

  # Conditional on previous job's output
  deploy-canary:
```

```
    needs: test
    if: |
      needs.test.result == 'success' &&
      github.event_name == 'push' &&
      startsWith(github.ref, 'refs/heads/release/')
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy-canary.sh

  # NOT skipped: only run on non-fork PRs (forks can't access secrets)
  security-scan:
    if: github.event.pull_request.head.repo.full_name == github.repository
    runs-on: ubuntu-latest
    steps:
      - run: ./security-scan.sh
```

The needs Context for Job Conditions

```
jobs:
  job-a:
    runs-on: ubuntu-latest
    outputs:
      status: ${{ steps.check.outputs.status }}
    steps:
      - id: check
        run: echo "status=ready" >> $GITHUB_OUTPUT

  job-b:
    runs-on: ubuntu-latest
    steps:
      - run: ./might-fail.sh

  job-c:
    needs: [job-a, job-b]
    # Access individual job results via needs.<job>.result
    # Possible values: success | failure | cancelled | skipped
    if: |
      always() &&
      needs.job-a.result == 'success' &&
      (needs.job-b.result == 'success' || needs.job-b.result == 'failure')
    # ^ Run if job-a succeeded AND job-b either passed or failed
    # (but not if job-b was cancelled or skipped)
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo "job-a: ${{ needs.job-a.result }}"
          echo "job-b: ${{ needs.job-b.result }}"
          echo "job-a output: ${{ needs.job-a.outputs.status }}"
```

Common Real-World Conditional Patterns

```yaml
jobs:

  # ── Only run on non-draft PRs ──────────────────────────────
  ci:
    if: github.event.pull_request.draft == false
    runs-on: ubuntu-latest
    steps:
      - run: npm test

  # ── Skip if commit message contains [skip ci] ──────────────
  build:
    if: "!contains(github.event.head_commit.message, '[skip ci]')"
    runs-on: ubuntu-latest
    steps:
      - run: make build

  # ── Run only on specific file changes ──────────────────────
  # (Simpler than path filters; uses github.event context)
  deploy-docs:
    if: |
      github.event_name == 'push' &&
      contains(github.event.head_commit.modified, 'docs/')
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy-docs.sh

  # ── Conditional step: different behavior per OS ────────────
  cross-platform-build:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Install dependencies (Linux/macOS)
        if: runner.os != 'Windows'
        run: sudo apt-get install -y libssl-dev || brew install openssl

      - name: Install dependencies (Windows)
        if: runner.os == 'Windows'
        run: choco install openssl

      - run: make build

  # ── Robust cleanup that always runs ────────────────────────
  integration-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
```

```
      - name: Start test environment
        id: setup
        run: docker-compose up -d

      - name: Run integration tests
        run: npm run test:integration

      # This cleanup runs even if tests fail
      - name: Teardown (always)
        if: always()
        run: docker-compose down -v

      # This notification runs only on CI failure on main
      - name: Notify on failure
        if: failure() && github.ref == 'refs/heads/main'
        run: |
          curl -X POST ${{ secrets.SLACK_WEBHOOK }} \
            -d '{"text":"Integration tests failed on main!"}'

  # ── Full pipeline with all condition types ──────────────────────
  pipeline:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Build          # Runs on success (default)
        id: build
        run: make build

      - name: Test           # Runs only if build succeeded
        id: test
        run: make test

      - name: Deploy         # Runs only if test succeeded
        if: success()
        run: ./deploy.sh

      - name: Rollback       # Runs only if deploy failed
        if: failure() && steps.build.outcome == 'success'
        # steps.<id>.outcome: success | failure | cancelled | skipped
        run: ./rollback.sh

      - name: Summary        # Always runs
        if: always()
        run: |
          echo "Build: ${{ steps.build.outcome }}"
          echo "Test: ${{ steps.test.outcome }}"
          echo "Overall: ${{ job.status }}"
          cat >> $GITHUB_STEP_SUMMARY << EOF
          ## Pipeline Results
          | Stage | Status |
          |-------|--------|
          | Build | ${{ steps.build.outcome }} |
```

```
            | Test   | ${{ steps.test.outcome }}   |
            EOF
```

The Difference Between `outcome` and `conclusion`

```yaml
# steps.<id>.outcome  -- the step's result BEFORE continue-on-error is applied
# steps.<id>.conclusion -- the step's result AFTER continue-on-error is applied

steps:
  - id: flaky-step
    continue-on-error: true   # Step "succeeds" even on failure
    run: ./flaky-test.sh      # Exits with code 1

  # outcome = failure  (what actually happened)
  # conclusion = success  (after continue-on-error applied)

  - if: steps.flaky-step.outcome == 'failure'
    run: echo "flaky step actually failed, but we continued"

  - if: steps.flaky-step.conclusion == 'success'
    run: echo "this ALSO runs because continue-on-error made it 'succeed'"
```

Interview Answer (30-45 seconds)

> "GitHub Actions conditions use the `if:` key at job or step level. The four status functions are: `success()` (default -- runs if previous succeeded), `failure()` (runs only if something failed), `always()` (runs regardless, including cancellation), and `cancelled()` (runs only if cancelled). At the job level, `if: failure()` evaluates against ALL `needs:` dependencies. At the step level, it evaluates against all previous steps in the job. The `needs.<job>.result` context gives you the exact result of each dependency -- `success`, `failure`, `cancelled`, or `skipped` -- for fine-grained conditional logic. Critical pattern: always use `if: always()` for cleanup steps like tearing down Docker Compose environments -- if the test step fails, a cleanup step with no `if:` is skipped, leaving orphaned resources."

Gotchas ⚠

- **`if:` at job level using `failure()` means ANY dependency failed.** If you have `needs: [job-a, job-b]` and both succeed, `if: failure()` is false. If job-b fails, it's true. If you want to check a specific job, use `needs.job-a.result == 'failure'` instead of `failure()`.
- **`if:` is implicitly an expression.** You do NOT wrap it in `${{ }}`. Writing `if: ${{ failure() }}` works but is redundant and inconsistent. Write `if: failure()` directly.
- **A skipped job's `result` is `'skipped'`, not `'success'`.** If a job is skipped due to its `if:` condition, downstream jobs checking `needs.my-job.result == 'success'` will get `false`. Use

`needs.my-job.result != 'failure'` if you want to treat skipped as acceptable.

- **`always()` does NOT make a job immune to concurrency cancellation.** If the entire workflow run is cancelled (by concurrency or manually), jobs marked `if: always()` that haven't started yet are still cancelled before starting. `always()` only helps for steps/jobs that were already provisioned and started.
- **`continue-on-error: true` makes a FAILED step appear as `conclusion: success`** in the downstream condition, masking actual failures. Use `steps.<id>.outcome` (not `conclusion`) when you need to know what actually happened.

5.6 Manual Approvals and workflow_dispatch with Inputs

What It Is

`workflow_dispatch` is a trigger that allows manually running a workflow from the GitHub UI, CLI, or API. It supports typed inputs that parameterize the run. Combined with GitHub Environments' required reviewers, it forms the complete manual approval and controlled execution model.

Jenkins equivalent: Jenkins `input()` step (for in-pipeline approval gates) and Jenkins' "Build with Parameters" for manual triggers. The difference: `workflow_dispatch` inputs are declared in YAML with full typing and validation; the GitHub UI auto-renders a form based on the input declarations.

---

workflow_dispatch Inputs — All Types

```yaml
name: Manual Operations

on:
  workflow_dispatch:
    inputs:

      # ── String input ──────────────────────────────────────
      service-name:
        description: 'Name of the service to deploy'
        required: true
        type: string
        default: 'payment-api'

      # ── Choice input (dropdown) ───────────────────────────
      environment:
        description: 'Target deployment environment'
        required: true
        type: choice
        options:
          - staging
          - canary
```

```yaml
          - production
        default: staging

      # ── Boolean input (checkbox) ──────────────────────────────
      dry-run:
        description: 'Perform a dry run without making changes'
        required: false
        type: boolean
        default: false

      # ── Number input ──────────────────────────────────────────
      replicas:
        description: 'Number of pod replicas to deploy'
        required: false
        type: number
        default: 3

      # ── Environment input (special: shows env protection gate) ─
      deploy-environment:
        description: 'GitHub environment to target'
        required: false
        type: environment
        # Populated from your repo's configured environments
        # When selected, environment protection rules are automatically
applied

permissions:
  id-token: write
  contents: read

jobs:
  validate-and-deploy:
    runs-on: ubuntu-latest
    environment: ${{ github.event.inputs.environment }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Log dispatch inputs
        run: |
          echo "Service:     ${{ github.event.inputs.service-name }}"
          echo "Environment: ${{ github.event.inputs.environment }}"
          echo "Dry run:     ${{ github.event.inputs.dry-run }}"
          echo "Replicas:    ${{ github.event.inputs.replicas }}"
          echo "Triggered by: ${{ github.actor }}"

      - name: Deploy
        run: |
          DRY_FLAG=""
          if [ "${{ github.event.inputs.dry-run }}" = "true" ]; then
            DRY_FLAG="--dry-run"
            echo "DRY RUN -- no changes applied"
          fi
```

```
            ./deploy.sh \
              --service ${{ github.event.inputs.service-name }} \
              --env ${{ github.event.inputs.environment }} \
              --replicas ${{ github.event.inputs.replicas }} \
              ${DRY_FLAG}
```

Triggering workflow_dispatch via CLI and API

```
# GitHub CLI -- interactive
gh workflow run deploy.yml

# GitHub CLI -- with inputs
gh workflow run deploy.yml \
  --ref main \
  -f service-name=payment-api \
  -f environment=production \
  -f dry-run=false \
  -f replicas=5

# GitHub CLI -- list recent runs to check status
gh run list --workflow=deploy.yml

# REST API
curl -X POST \
  -H "Authorization: Bearer $GH_TOKEN" \
  -H "Accept: application/vnd.github.v3+json" \
  "https://api.github.com/repos/my-org/my-
repo/actions/workflows/deploy.yml/dispatches" \
  -d '{
    "ref": "main",
    "inputs": {
      "service-name": "payment-api",
      "environment": "production",
      "dry-run": "false",
      "replicas": "5"
    }
  }'
```

Advanced workflow_dispatch Patterns

Pattern 1: Manual Rollback Workflow

```
name: Emergency Rollback

on:
  workflow_dispatch:
    inputs:
```

```yaml
      service:
        description: 'Service to roll back'
        required: true
        type: choice
        options: [payment-api, user-service, inventory-service, order-
service]

      environment:
        description: 'Environment to roll back'
        required: true
        type: choice
        options: [staging, production]

      target-revision:
        description: 'Helm revision to roll back to (leave blank for
previous)'
        required: false
        type: string
        default: ''

      reason:
        description: 'Reason for rollback (included in audit log)'
        required: true
        type: string

permissions:
  id-token: write
  contents: read

jobs:
  rollback:
    runs-on: ubuntu-latest
    environment: ${{ github.event.inputs.environment }}   # Triggers
approval gate if production
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Log rollback initiation
        run: |
          cat >> $GITHUB_STEP_SUMMARY << EOF
          ## Emergency Rollback
          - **Service:** ${{ github.event.inputs.service }}
          - **Environment:** ${{ github.event.inputs.environment }}
          - **Initiated by:** ${{ github.actor }}
          - **Reason:** ${{ github.event.inputs.reason }}
          - **Time:** $(date -u +"%Y-%m-%dT%H:%M:%SZ")
          EOF

      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.DEPLOY_ROLE_ARN }}
          aws-region: us-east-1
```

```
      - name: Execute rollback
        run: |
          SERVICE="${{ github.event.inputs.service }}"
          ENV="${{ github.event.inputs.environment }}"
          REVISION="${{ github.event.inputs.target-revision }}"

          if [ -n "$REVISION" ]; then
            helm rollback "$SERVICE" "$REVISION" --namespace "$ENV" --wait
          else
            helm rollback "$SERVICE" --namespace "$ENV" --wait
          fi

          echo "Rolled back to: $(helm history $SERVICE --namespace $ENV -
-max 1 --output json | jq -r '.[0].revision')"
```

Pattern 2: Parameterized Batch Operation

```
name: Database Migration

on:
  workflow_dispatch:
    inputs:
      migration-version:
        description: 'Target migration version (e.g., V20240315_001)'
        required: true
        type: string

      environment:
        type: choice
        options: [staging, production]
        required: true

      backup-first:
        description: 'Create database backup before migrating'
        type: boolean
        default: true

      dry-run:
        description: 'Preview migration without applying changes'
        type: boolean
        default: true

jobs:
  migrate:
    runs-on: ubuntu-latest
    environment: ${{ github.event.inputs.environment }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Backup database
        if: github.event.inputs.backup-first == 'true'
```

```
        run: |
          ./db-backup.sh \
            --env ${{ github.event.inputs.environment }} \
            --label "pre-migration-${{ github.event.inputs.migration-
version }}"

      - name: Run migration
        run: |
          ARGS="--target ${{ github.event.inputs.migration-version }}"
          if [ "${{ github.event.inputs.dry-run }}" = "true" ]; then
            ARGS="$ARGS --dry-run"
          fi
          ./flyway.sh migrate $ARGS

      - name: Verify migration
        if: github.event.inputs.dry-run == 'false'
        run: ./flyway.sh validate
```

Pattern 3: Multi-Stage with Intermediate Approval (Environment Gate)

```
# workflow_dispatch + environment protection = manual trigger + approval
gate
name: Production Promotion

on:
  workflow_dispatch:
    inputs:
      image-tag:
        description: 'Image tag to promote to production'
        required: true
        type: string

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - name: Verify image exists in staging
        run: |
          docker pull ghcr.io/my-org/myapp:${{ github.event.inputs.image-
tag }}
          echo "Image verified"

  # This job PAUSES pending approval before runner starts
  promote-to-production:
    needs: validate
    runs-on: ubuntu-latest
    environment: production    # Required reviewers configured here
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: |
          echo "Promoting ${{ github.event.inputs.image-tag }} to
```

```
production"
          ./promote.sh ${{ github.event.inputs.image-tag }}
```

Interview Answer (30-45 seconds)

> "`workflow_dispatch` enables manually triggering workflows from the GitHub UI, CLI, or API with typed inputs. Input types are string, boolean, number, choice (dropdown), and environment (dropdown populated from your configured environments). When you declare a `choice` or `environment` input, GitHub renders the form automatically in the UI — no custom tooling needed. For approval gates combined with manual triggers, the pattern is `workflow_dispatch` for the trigger plus `environment:` on the target job — the environment's required reviewers provide the approval gate. This is how you build controlled manual operations: a rollback workflow with a reason field, a migration workflow with a dry-run option, or a promotion workflow that requires an ops team member to approve before production changes."

Gotchas ⚠

- **All `workflow_dispatch` inputs arrive as strings regardless of declared type.** `type: boolean` with `default: false` arrives as the string `"false"`. In shell: `if [ "${{ github.event.inputs.dry-run }}" = "true" ]`. In expressions: `github.event.inputs.dry-run == 'true'` (string comparison).
- **`workflow_dispatch` only appears for workflows on the default branch** (usually `main`) in the GitHub UI's Actions tab. If the workflow is only on a feature branch, you must use the CLI or API to trigger it.
- **The `type: environment` input shows only environments that exist in the repo.** If you haven't created the environment yet, it won't appear in the dropdown. Create environments first in Repo Settings.
- **`github.event.inputs` and `inputs` context are both available in `workflow_dispatch`.** Use `inputs.` (shorter) in workflow expressions. In reusable workflows called via `workflow_call`, only `inputs.` is valid.
- **There is no built-in audit trail for WHY a dispatch was triggered** beyond the actor name. Use a `reason` string input and write it to `$GITHUB_STEP_SUMMARY` as shown in the rollback example above.

5.7 Dynamic Workflow Generation — Generating YAML at Runtime

What It Is

Dynamic workflow generation is the pattern of creating, modifying, or interpreting workflow definitions at runtime — either generating YAML on-the-fly and dispatching it via the API, or using

`repository_dispatch` / `workflow_dispatch` to trigger parameterized workflows with computed inputs.

This is the most advanced workflow pattern and is asked in staff-level interviews at companies with large-scale CI/CD requirements.

Jenkins equivalent: Jenkins Job DSL plugin, dynamically generated Jenkinsfiles, or programmatic use of the Jenkins REST API to create/trigger jobs. The difference: GitHub Actions workflows live in `.github/workflows/` and are not directly mutable at runtime — you work around this using dispatch, reusable workflows, and matrix patterns.

Why Dynamic Generation Is Needed

```
Problems that require dynamic solutions:

1. Monorepo with 100 services:
   - Static matrix: hardcode all 100 services -- unmaintainable
   - Dynamic matrix: detect changed services at runtime -- solved in 5.1

2. Generated test plans:
   - A pre-analysis step determines which test suites to run
   - Number and composition of suites changes per commit
   - Static matrix: can't know at YAML-write time what tests exist

3. Cross-repo orchestration:
   - A release workflow needs to trigger deployments in 20 service repos
   - Each repo has slightly different parameters
   - Need to generate and fire N workflow dispatch calls

4. Config-driven pipelines:
   - Teams provide a config file in their repo (pipeline.yaml)
   - A platform workflow reads it and creates the appropriate pipeline
steps
   - Different teams get different steps based on their config
```

Pattern 1: repository_dispatch — API-Triggered Workflows

```yaml
# Receiving workflow (in target repo)
# .github/workflows/receive-dispatch.yml
name: Receive Repository Dispatch

on:
  repository_dispatch:
    types:
      - deploy-service      # Custom event type
      - run-integration-tests
      - promote-to-env
```

```yaml
jobs:
  handle:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Route by event type
        run: |
          EVENT="${{ github.event.action }}"
          echo "Received dispatch: $EVENT"
          echo "Payload: ${{ toJSON(github.event.client_payload) }}"

          case "$EVENT" in
            deploy-service)
              ./deploy.sh \
                "${{ github.event.client_payload.service }}" \
                "${{ github.event.client_payload.image_tag }}" \
                "${{ github.event.client_payload.environment }}"
              ;;
            promote-to-env)
              ./promote.sh \
                "${{ github.event.client_payload.from_env }}" \
                "${{ github.event.client_payload.to_env }}"
              ;;
            *)
              echo "Unknown event: $EVENT"
              exit 1
              ;;
          esac
```

```bash
# Sending the dispatch (from another system, workflow, or script)
# client_payload can be any JSON up to 10 KB

# From a workflow in another repo:
curl -X POST \
  -H "Authorization: Bearer ${{ secrets.ORG_DISPATCH_TOKEN }}" \
  -H "Accept: application/vnd.github.v3+json" \
  "https://api.github.com/repos/my-org/payment-api/dispatches" \
  -d '{
    "event_type": "deploy-service",
    "client_payload": {
      "service": "payment-api",
      "image_tag": "abc123",
      "environment": "staging",
      "triggered_by": "'"$GITHUB_ACTOR"'",
      "source_run_id": "'"$GITHUB_RUN_ID"'"
    }
  }'
```

Pattern 2: Orchestrator Workflow — Trigger N Child Workflows

```
# An orchestrator that reads a manifest and fires per-service deployments
name: Orchestrated Multi-Service Deploy

on:
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [staging, production]
      services-json:
        description: 'JSON array of services to deploy (leave empty for
all)'
        type: string
        default: '[]'

jobs:
  orchestrate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Determine services to deploy
        id: services
        run: |
          REQUESTED='${{ github.event.inputs.services-json }}'

          if [ "$REQUESTED" = "[]" ] || [ -z "$REQUESTED" ]; then
            # Deploy ALL services from the manifest
            SERVICES=$(cat deploy/manifest.json | jq -c '[.[].name]')
          else
            SERVICES="$REQUESTED"
          fi

          echo "services=${SERVICES}" >> $GITHUB_OUTPUT
          echo "Deploying: $SERVICES"

      - name: Dispatch deployment for each service
        run: |
          ENV="${{ github.event.inputs.environment }}"
          SERVICES='${{ steps.services.outputs.services }}'

          echo "$SERVICES" | jq -r '.[]' | while read SERVICE; do
            CONFIG=$(cat deploy/manifest.json | jq -c ".[] | select(.name
== \"$SERVICE\")")
            IMAGE_TAG=$(echo "$CONFIG" | jq -r '.current_tag')

            echo "Dispatching deploy for $SERVICE (tag: $IMAGE_TAG)"

            # Trigger workflow dispatch in each service repo
            gh workflow run deploy.yml \
```

```
              --repo "my-org/${SERVICE}" \
              --ref main \
              -f environment="$ENV" \
              -f image-tag="$IMAGE_TAG" \
              -f triggered-by="orchestrator/${{ github.run_id }}"

            sleep 2    # Avoid rate limiting
          done
        env:
          GH_TOKEN: ${{ secrets.ORG_WORKFLOW_DISPATCH_TOKEN }}

      - name: Wait for all deployments to complete
        run: |
          # Poll status of dispatched workflows
          ENV="${{ github.event.inputs.environment }}"
          SERVICES='${{ steps.services.outputs.services }}'
          TIMEOUT=1800    # 30 minutes
          START=$(date +%s)

          echo "$SERVICES" | jq -r '.[]' | while read SERVICE; do
            while true; do
              NOW=$(date +%s)
              ELAPSED=$((NOW - START))

              if [ $ELAPSED -gt $TIMEOUT ]; then
                echo "::error::Timeout waiting for $SERVICE deploy"
                exit 1
              fi

              STATUS=$(gh run list \
                --repo "my-org/${SERVICE}" \
                --workflow deploy.yml \
                --limit 1 \
                --json status,conclusion \
                --jq '.[0]')

              CONCLUSION=$(echo "$STATUS" | jq -r '.conclusion // empty')

              if [ "$CONCLUSION" = "success" ]; then
                echo "$SERVICE: deployed successfully"
                break
              elif [ "$CONCLUSION" = "failure" ] || [ "$CONCLUSION" =
"cancelled" ]; then
                echo "::error::$SERVICE deployment $CONCLUSION"
                exit 1
              else
                echo "$SERVICE: still running... (${ELAPSED}s elapsed)"
                sleep 30
              fi
            done
          done
        env:
          GH_TOKEN: ${{ secrets.ORG_WORKFLOW_DISPATCH_TOKEN }}
```

Pattern 3: Config-Driven Pipeline (Reading pipeline.yaml)

```
# App repos provide a pipeline config:
# .pipeline.yml
# steps:
#   - name: lint
#     command: npm run lint
#   - name: test
#     command: npm test
#     coverage: true
#   - name: e2e
#     command: npm run e2e
#     optional: true
#   docker:
#     enabled: true
#     registry: ghcr.io/my-org
#   environments:
#     - name: staging
#       auto-deploy: true
#     - name: production
#       auto-deploy: false
#       require-approval: true

name: Config-Driven Pipeline

on:
  push:
    branches: [main]
  pull_request:

jobs:
  read-config:
    runs-on: ubuntu-latest
    outputs:
      config: ${{ steps.parse.outputs.config }}
      run-e2e: ${{ steps.parse.outputs.run-e2e }}
      docker-enabled: ${{ steps.parse.outputs.docker-enabled }}
      environments: ${{ steps.parse.outputs.environments }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Parse pipeline config
        id: parse
        run: |
          if [ ! -f .pipeline.yml ]; then
            # Default config if none provided
            echo 'config={"steps":[{"name":"build","command":"make
build"},{"name":"test","command":"make test"}],"docker":
{"enabled":false},"environments":[]}' >> $GITHUB_OUTPUT
            echo "run-e2e=false" >> $GITHUB_OUTPUT
```

```
            echo "docker-enabled=false" >> $GITHUB_OUTPUT
            echo "environments=[]" >> $GITHUB_OUTPUT
            exit 0
          fi

          # Parse YAML to JSON using Python (yq/jq not always available)
          CONFIG=$(python3 -c "
          import yaml, json, sys
          with open('.pipeline.yml') as f:
            config = yaml.safe_load(f)
          print(json.dumps(config))
          ")

          echo "config=${CONFIG}" >> $GITHUB_OUTPUT

          # Extract specific values for job conditions
          RUN_E2E=$(echo "$CONFIG" | jq 'any(.steps[]; .name == "e2e")')
          DOCKER=$(echo "$CONFIG" | jq '.docker.enabled // false')
          ENVS=$(echo "$CONFIG" | jq -c '[.environments[].name]')

          echo "run-e2e=${RUN_E2E}" >> $GITHUB_OUTPUT
          echo "docker-enabled=${DOCKER}" >> $GITHUB_OUTPUT
          echo "environments=${ENVS}" >> $GITHUB_OUTPUT

  # Run steps defined in config
  run-steps:
    needs: read-config
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Execute configured steps
        run: |
          CONFIG='${{ needs.read-config.outputs.config }}'

          echo "$CONFIG" | jq -r '.steps[] | @base64' | while read
STEP_B64; do
            STEP=$(echo "$STEP_B64" | base64 -d)
            NAME=$(echo "$STEP" | jq -r '.name')
            CMD=$(echo "$STEP" | jq -r '.command')
            OPTIONAL=$(echo "$STEP" | jq -r '.optional // false')

            echo "::group::Step: $NAME"
            echo "Running: $CMD"

            if eval "$CMD"; then
              echo "Step $NAME: SUCCESS"
            elif [ "$OPTIONAL" = "true" ]; then
              echo "::warning::Optional step $NAME failed -- continuing"
            else
              echo "::error::Required step $NAME failed"
              exit 1
            fi
            echo "::endgroup::"
```

```
          done

  # Build Docker image if config says so
  docker-build:
    needs: [read-config, run-steps]
    if: needs.read-config.outputs.docker-enabled == 'true'
    runs-on: ubuntu-latest
    permissions:
      packages: write
      id-token: write
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - name: Build and push
        run: |
          REGISTRY=$(echo '${{ needs.read-config.outputs.config }}' | jq -
r '.docker.registry')
          docker build -t "${REGISTRY}/${{ github.event.repository.name
}}:${{ github.sha }}" .

  # Deploy to each configured environment
  deploy-environments:
    needs: [read-config, docker-build]
    if: |
      github.event_name == 'push' &&
      needs.read-config.outputs.environments != '[]'
    strategy:
      matrix:
        environment: ${{ fromJSON(needs.read-config.outputs.environments)
}}
    runs-on: ubuntu-latest
    environment: ${{ matrix.environment }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: |
          AUTO_DEPLOY=$(echo '${{ needs.read-config.outputs.config }}' \
            | jq -r --arg env "${{ matrix.environment }}" \
            '.environments[] | select(.name == $env) | ."auto-deploy"')

          if [ "$AUTO_DEPLOY" = "true" ]; then
            ./deploy.sh ${{ matrix.environment }} ${{ github.sha }}
          else
            echo "::notice::${{ matrix.environment }} requires manual
approval -- skipping auto-deploy"
```

Pattern 4: Generating and Dispatching Workflow YAML

```
# Advanced: generate a workflow YAML file and commit + push
# (forces new workflow run via the auto-generated file)
# Use case: nightly auto-generated test plan based on test discovery
```

```yaml
name: Generate Test Plan

on:
  schedule:
    - cron: '0 0 * * *'    # Nightly
  workflow_dispatch:

permissions:
  contents: write

jobs:
  generate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Discover tests and generate workflow
        run: |
          # Discover all test suite files
          SUITES=$(find tests/ -name "suite_*.py" | sort | \
                   xargs -I{} basename {} .py | \
                   sed 's/suite_//')

          # Generate the workflow YAML
          cat > .github/workflows/generated-test-run.yml << 'WORKFLOW_EOF'
          name: Generated Nightly Tests
          on:
            workflow_dispatch:
          jobs:
          WORKFLOW_EOF

          echo "$SUITES" | while read SUITE; do
            cat >> .github/workflows/generated-test-run.yml << EOF
            test-${SUITE}:
              runs-on: ubuntu-latest
              steps:
                - uses:
actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
                - run: python -m pytest tests/suite_${SUITE}.py -v
          EOF
          done

      - name: Commit generated workflow
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "github-
actions[bot]@users.noreply.github.com"
          git add .github/workflows/generated-test-run.yml
          git diff --staged --quiet || \
            git commit -m "chore: regenerate nightly test workflow [skip
ci]"
          git push
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

```
      - name: Trigger the generated workflow
        run: |
          sleep 5    # Wait for GitHub to index the new commit
          gh workflow run generated-test-run.yml --ref main
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Interview Answer (30-45 seconds)

> "Dynamic workflow generation typically falls into three patterns. First, dynamic matrix: a pre-flight job computes which services changed or which tests to run, outputs the list as JSON, and a downstream job uses `fromJSON()` to drive the matrix — this is the most common pattern and covered earlier. Second, orchestration via dispatch: an orchestrator workflow reads a manifest and fires individual `workflow_dispatch` or `repository_dispatch` events for each target, then polls their status. Third, config-driven pipelines: app teams provide a `pipeline.yml` config file, a platform workflow reads it, and conditionally enables or disables jobs based on what the config declares. The genuine last resort is generating a YAML file and committing it, then triggering the generated workflow — but this is complex and should be a last resort after exhausing matrix and dispatch patterns."

Gotchas ⚠

- **You cannot modify a currently-running workflow's behavior at runtime.** Workflow YAML is read once at dispatch time. Generating YAML and triggering a new run is a workaround — not modifying the running one.
- **`repository_dispatch` requires a PAT or GitHub App token, NOT `GITHUB_TOKEN`.** `GITHUB_TOKEN` cannot trigger `repository_dispatch` in other repos. Use an org-level PAT with `repo` scope or a GitHub App token.
- **Polling for dispatched workflow completion has no native wait primitive.** You poll via the API. Always implement a timeout to prevent infinite loops.
- **Generated YAML committed to `.github/workflows/` with `GITHUB_TOKEN` does NOT trigger a new workflow run** — GITHUB_TOKEN pushes intentionally don't trigger workflow events. Use a PAT or GitHub App token for the commit, or explicitly call `gh workflow run` after committing.
- **`client_payload` in `repository_dispatch` is limited to 10 KB.** For larger data sets, write the data to a file in a known location (S3, artifact storage) and pass only the reference in the payload.
- **Rate limits apply to API calls in orchestrator workflows.** GitHub's secondary rate limit kicks in if you fire more than ~20 API requests per minute from a single token. Add `sleep 2` between dispatch calls and implement exponential backoff for the polling loop.

Cross-Topic Interview Questions

Q: You have a monorepo with 50 services. Every PR rebuild tests all 50 services even if only one changed. How do you fix this?

> "Dynamic matrix from path detection. A pre-flight job runs `git diff --name-only` between the PR's head commit and the base branch. It parses the output to extract service names from changed file paths (anything under `services/<name>/`), serializes the list as a JSON array to a job output, and a downstream test job uses `fromJSON(needs.detect.outputs.services)` as its matrix. Only the changed services get parallel test jobs. For robustness: if the diff includes shared library changes (`lib/` or `common/`), default to running all services since a shared change could affect anything. Two gotchas to address: empty matrix errors if nothing in `services/` changed (guard with `if: needs.detect.outputs.has-changes == 'true'`), and the 256-matrix-combination limit for extremely large repos."

Q: Two engineers push to main 30 seconds apart. Both trigger the deploy pipeline. What happens and how do you prevent race conditions?

> "Without concurrency controls, both deploys run simultaneously. They race to update the same Kubernetes deployment or Helm release — one will succeed, one may partially apply, and you can end up in an undefined intermediate state. The fix is `concurrency:` on the deploy job with `cancel-in-progress: false` and a group name scoped to the environment: `group: deploy-production-${{ github.repository }}`. With this config, when the second deploy arrives while the first is running, it waits in the queue. Critically: `cancel-in-progress: false` is mandatory for deploys — you never want to cancel a running deployment midway. If a third push arrives while both the first is running and the second is waiting, the second waiting run is cancelled and the third takes its place in the queue — only ever one pending deploy per environment."

Q: You want CI to run on PRs from forks but also post status comments on those PRs. You can't do both in one workflow — why, and what's the solution?

> "Fork PRs triggering `pull_request` don't have access to secrets — GitHub intentionally removes them for security. Secrets are needed to post a PR comment using the GITHUB_TOKEN with `pull-requests: write`. So you can't run untrusted fork code AND post comments in the same workflow context. The two-workflow solution: Workflow 1 uses `pull_request` trigger, runs the tests with no secrets, and uploads the test results as an artifact. Workflow 2 uses `workflow_run` trigger referencing Workflow 1 by name, runs in the base-branch context with full secrets, downloads the artifact from the completed run using `run-id: ${{ github.event.workflow_run.id }}`, and posts the results as a PR comment. The key: Workflow 2 never executes any code from the PR — it only reads pre-built artifacts. Always check `github.event.workflow_run.conclusion == 'success'` before posting."

Quick Reference Cheat Sheet

Matrix

```yaml
# Static
strategy:
  fail-fast: false
  max-parallel: 4
  matrix:
    os: [ubuntu-latest, windows-latest]
    version: [18, 20, 22]
    include:
      - os: macos-latest
        version: 20
    exclude:
      - os: windows-latest
        version: 18

# Dynamic -- two-job pattern
detect-job:
  outputs:
    items: ${{ steps.out.outputs.items }}
  steps:
    - id: out
      run: echo "items=$(compute | jq -c .)" >> $GITHUB_OUTPUT

use-matrix:
  needs: detect-job
  if: needs.detect-job.outputs.items != '[]'
  strategy:
    matrix:
      item: ${{ fromJSON(needs.detect-job.outputs.items) }}
```

Concurrency

```yaml
# Cancel duplicates (CI): cancel-in-progress: true
# Queue deploys: cancel-in-progress: false
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}
```

Cache Key Formula

```yaml
key: ${{ runner.os }}-<ecosystem>-${{ hashFiles('**/lockfile') }}
restore-keys: |
  ${{ runner.os }}-<ecosystem>-
```

Condition Quick Reference

```
if: success()      # Default -- previous steps/jobs succeeded
if: failure()      # Previous step/job failed
if: always()       # Regardless of status (cleanup, summaries)
if: cancelled()    # Workflow was cancelled

# Job condition using needs:
if: needs.my-job.result == 'success'
if: always() && needs.job-a.result == 'success'

# Step outcome vs conclusion:
steps.<id>.outcome     # Actual result before continue-on-error
steps.<id>.conclusion  # Result after continue-on-error applied
```

Artifacts vs Cache vs Outputs

```
Job outputs:  Small values (strings < 50KB) between jobs
Artifacts:    Files between jobs; per-run; download after run
Cache:        Files between RUNS; keyed by hash; dependency caching
```

workflow_dispatch Input Types

```
inputs:
  name:    { type: string,  required: true,  default: 'val' }
  flag:    { type: boolean, required: false, default: false }
  count:   { type: number,  required: false, default: 3 }
  env:     { type: choice,  options: [a, b, c] }
  target:  { type: environment }    # Populated from repo environments
# All arrive as strings in shell -- compare with == 'true', not == true
```

Dynamic Dispatch

```
# workflow_dispatch via CLI
gh workflow run deploy.yml --ref main -f env=production -f tag=abc123

# repository_dispatch (requires PAT, not GITHUB_TOKEN)
curl -X POST \
  -H "Authorization: Bearer $PAT" \
  "https://api.github.com/repos/ORG/REPO/dispatches" \
  -d '{"event_type":"deploy","client_payload":
{"service":"api","tag":"v1.2"}}'
```

End of Category 5: Advanced Workflow Patterns

> **Next:** Category 6 (Self-Hosted Runners & ARC) builds directly on everything here — matrix strategies interact with runner labels, concurrency controls are critical for shared self-hosted runner pools, and caching behavior differs fundamentally when runners are persistent rather than ephemeral.
>
> **Priority drill topics from this category:**
>
> - Dynamic matrix: be able to write the full two-job detect-then-matrix pattern from memory including the empty-matrix guard
> - Concurrency: explain the "only one run waits" queue semantics, and the cancel vs queue decision for CI vs deployments
> - Cache keys: write a correct key formula from scratch for npm and explain why hashFiles(lockfile) is the right anchor
> - `pull_request_target` + artifacts two-workflow pattern for fork PR CI